

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**



ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И УПРАВЛЯЮЩИХ СИСТЕМ

Лабораторная работа №2
по дисциплине: Операционные системы
тема: «**Виртуальная память в ОС Linux**»

Выполнил: ст. группы ВТ-231
Ковалев А. В.

Проверили:
Островский А. М.

Белгород 2025 г

Лабораторная работа № 2

Цель работы: Изучить концепцию виртуальной памяти в Linux, механизмы отображения виртуальных адресов в физические, организацию памяти процесса, работу подкачки и кэширования.

Цель работы обуславливает постановку и решение следующих задач:

1. Изучить концепцию виртуальной памяти в ОС Linux: назначение, преимущества, отличие от физической памяти.
2. Ознакомиться с механизмами отображения виртуальных адресов в физические.
3. Рассмотреть организацию адресного пространства процесса: сегменты кода, данных, кучи, стека, область динамически подгружаемых библиотек (shared libraries).
4. Освоить использование системных вызовов для управления памятью:
 - 4.1. `mmap()`, `munmap()` — отображение файлов и анонимное отображение в память.
 - 4.2. `mlock()`, `munlock()`, `mlockall()`, `munlockall()` — блокировка страниц в памяти, предотвращение выгрузки в swar.
 - 4.3. `mprotect()` — изменение прав доступа к страницам (чтение/запись/исполнение).
 - 4.4. `madvise()` — передача советов ядру по предполагаемым шаблонам доступа (например, `MADV_SEQUENTIAL`, `MADV_RANDOM`, `MADV_DONTNEED`). `posix_madvise()` — аналогичный механизм в POSIX.
5. Изучить организацию взаимодействия функций стандартной библиотеки для динамического выделения памяти (`malloc()`, `calloc()`, `realloc()`, `free()`) с системными вызовами (`brk()`, `mmap()`).
6. Ознакомиться с механизмами работы ядра с памятью:
 - 6.1. Копирование-при-записи (Copy-on-Write, COW) — экономия памяти при порождении процессов через `fork()`, создание копий страниц только при изменении.
 - 6.2. Механизм подкачки страниц — перенос страниц из ОЗУ во вторичное хранилище (swar) и обратно; причины и последствия подкачки.
 - 6.3. Кэш страниц — роль в ускорении операций чтения / записи файлов, стратегии замещения страниц.
 - 6.4. Обработка отказов по странице (page fault): причины (отсутствие страницы в памяти, защита доступа) и реакции ядра.
7. Освоить консольные инструменты для анализа состояния памяти.
 - 7.1. `free` — просмотр использования оперативной памяти и swar-раздела.
 - 7.2. `vmstat` — мониторинг состояния памяти, подкачки и нагрузки на систему.
 - 7.3. `top` / `htop` — контроль за использованием памяти процессами.
 - 7.4. `/proc/[PID]/maps` и `/proc/[PID]/smaps` — исследование структуры адресного пространства процесса, распределение анонимной и файловой памяти.
 - 7.5. `/proc/meminfo` — сводная информация об использовании памяти в системе.
 - 7.6. `ptop` — наглядное отображение карт памяти для процесса.
8. Выполнить индивидуальное задание для закрепления знаний на практике, подготовив соответствующую программу на языке C (номер задания соответствует номеру студента по журналу; если этот номер больше, чем максимальное число заданий, тогда вариант задания вычисляется по формуле: номер по журналу % максимальный номер задания, где % — остаток от деления; если остаток равен 0, студенту назначается задание с максимальным номером).
9. Подготовить отчёт по работе, который должен включать: краткое описание всех использованных системных вызовов и команд; текст программы; протоколы; скриншоты с демонстрацией поведения ОС Linux; выводы.

Вариант 2

Задание «Великий визирь». Подготовить обычный файл (например, /tmp/nums.bin) длины N (длина должна быть кратна размеру страницы памяти). Заполнить его случайными 64-битными

целыми числами. При помощи mmap() отобразить файл в адресное пространство процесса. Вычислить сумму квадратов элементов массива (сумма рассчитывается по модулю 264) двумя

способами: а) путем последовательного доступа к элементам массива; б) путем рандомизированного доступа к элементам. Для полного рандомизированного перебора элементов без повторений использовать либо случайную перестановку индексов, сформированную алгоритмом Фишера–Йетса, либо память-экономное перемешивание. Выяснить, как «подсказки» ядру влияют на стандартную работу алгоритмов: а) в режиме MADV_SEQUENTIAL, когда ядро будет заранее подкачивать страницы «вперёд по ходу чтения»; б) в режиме MADV_RANDOM, когда можно отключить избыточное опережающее чтение, которое бесполезно при хаотичном доступе; в) в режиме MADV_WILLNEED, когда ядро

может подкачать страницы в память заранее. Подсчитать число minor/major page faults и промахов кеша с помощью утилиты perf. Например, для 5 прогонов можно использовать такую

команду:

```
sudo perf stat -r 5 -e task-clock, instructions, cycles, cache-misses, \
minor-faults,major-faults ./lab2_c.o
```

Выполнение индивидуального задания:

Исходный код программы:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <time.h>

uint64_t rand_uint64() {
    uint64_t r = 0;
    for (int i = 0; i < 64; i += 8) {
        r |= ((uint64_t)rand() & 0xFF) << i;
    }
    return r;
}

void shuffle(size_t *array, size_t n) {
    for (size_t i = n - 1; i > 0; i--) {
        size_t j = rand() % (i + 1);
```

```

        size_t temp = array[i];
        array[i] = array[j];
        array[j] = temp;
    }
}

size_t calculate_file_size(size_t size_mb, long page_size) {
    size_t N = size_mb * 1024ULL * 1024;
    return (N / page_size) * page_size;
}

void generate_data_file(const char *filename, size_t file_size) {
    int fd = open(filename, O_RDWR | O_CREAT | O_TRUNC, 0666);
    if (fd == -1) {
        perror("open");
        exit(1);
    }

    if (ftruncate(fd, file_size) == -1) {
        perror("ftruncate");
        close(fd);
        exit(1);
    }

    uint64_t *fill_map = mmap(NULL, file_size, PROT_WRITE, MAP_SHARED, fd, 0);
    if (fill_map == MAP_FAILED) {
        perror("mmap for fill");
        close(fd);
        exit(1);
    }

    size_t num_elements = file_size / sizeof(uint64_t);
    for (size_t i = 0; i < num_elements; i++) {
        fill_map[i] = rand_uint64();
    }

    if (msync(fill_map, file_size, MS_SYNC) == -1) {
        perror("msync");
    }

    if (munmap(fill_map, file_size) == -1) {
        perror("munmap fill");
    }

    close(fd);
}

uint64_t *map_file(const char *filename, size_t file_size) {
    int fd = open(filename, O_RDWR);
    if (fd == -1) {
        perror("open for mapping");
    }

```

```

        exit(1);
    }

    uint64_t *map = mmap(NULL, file_size, PROT_READ, MAP_SHARED, fd, 0);
    if (map == MAP_FAILED) {
        perror("mmap for compute");
        close(fd);
        exit(1);
    }

    close(fd);
    return map;
}

void apply_madvise(uint64_t *map, size_t file_size, const char *advice_str) {
    int advice = -1;
    if (strcmp(advice_str, "seq") == 0) {
        advice = MADV_SEQUENTIAL;
    } else if (strcmp(advice_str, "rand") == 0) {
        advice = MADV_RANDOM;
    } else if (strcmp(advice_str, "will") == 0) {
        advice = MADV_WILLNEED;
    } else if (strcmp(advice_str, "none") != 0) {
        fprintf(stderr, "Invalid advice\n");
        exit(1);
    }

    if (advice != -1 && madvise(map, file_size, advice) == -1) {
        perror("madvise");
    }
}

uint64_t sequential_access(uint64_t *map, size_t num_elements) {
    uint64_t sum = 0;
    for (size_t i = 0; i < num_elements; i++) {
        uint64_t val = map[i];
        sum += val * val;
    }
    return sum;
}

uint64_t random_access(uint64_t *map, size_t num_elements) {
    size_t *indices = malloc(num_elements * sizeof(size_t));
    if (indices == NULL) {
        fprintf(stderr, "Cannot allocate indices array\n");
        exit(1);
    }

    for (size_t i = 0; i < num_elements; i++) {
        indices[i] = i;
    }
}

```

```

    shuffle(indices, num_elements);

    uint64_t sum = 0;
    for (size_t i = 0; i < num_elements; i++) {
        uint64_t val = map[indices[i]];
        sum += val * val;
    }

    free(indices);
    return sum;
}

double measure_time(uint64_t (*access_func)(uint64_t *, size_t), uint64_t *map,
size_t num_elements) {
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    uint64_t sum = access_func(map, num_elements);

    clock_gettime(CLOCK_MONOTONIC, &end);
    double time_taken = (end.tv_sec - start.tv_sec) + (end.tv_nsec -
start.tv_nsec) / 1e9;
    printf("Sum: %llu\n", (unsigned long long)sum);
    printf("Time taken: %.6f seconds\n", time_taken);

    return time_taken;
}

int main(int argc, char *argv[]) {
    if (argc != 4) {
        fprintf(stderr, "Usage: %s <size_mb> <access_mode: seq/rand> <advice:
none/seq/rand/will>\n", argv[0]);
        return 1;
    }

    size_t size_mb = strtoull(argv[1], NULL, 10);
    char *access_mode = argv[2];
    char *advice_str = argv[3];

    srand(time(NULL));

    long page_size = sysconf(_SC_PAGE_SIZE);
    if (page_size == -1) {
        perror("sysconf");
        return 1;
    }

    size_t file_size = calculate_file_size(size_mb, page_size);
    if (file_size == 0) {
        fprintf(stderr, "Size too small\n");
    }
}

```

```

        return 1;
    }

    size_t num_elements = file_size / sizeof(uint64_t);

    const char *filename = "/tmp/nums.bin";
    generate_data_file(filename, file_size);

    uint64_t *map = map_file(filename, file_size);
    apply_madvise(map, file_size, advice_str);

    uint64_t (*access_func)(uint64_t *, size_t);
    if (strcmp(access_mode, "seq") == 0) {
        access_func = sequential_access;
    } else if (strcmp(access_mode, "rand") == 0) {
        access_func = random_access;
    } else {
        fprintf(stderr, "Invalid access mode\n");
        munmap(map, file_size);
        return 1;
    }

    measure_time(access_func, map, num_elements);

    if (munmap(map, file_size) == -1) {
        perror("munmap");
    }

    return 0;
}

```

Результат выполнения программы:

```

Sum: 8834830041030240239
Time taken: 0.036828 seconds
Sum: 837965738272720748
Time taken: 0.036107 seconds
Sum: 14410614168697001953
Time taken: 0.037301 seconds
Sum: 6137988226362533523
Time taken: 0.036059 seconds
Sum: 3704910145454594185
Time taken: 0.035437 seconds

Performance counter stats for './lab2 100 seq none' (5 runs):

      944,93 msec task-clock                #   0,996 CPUs utilized          ( +- 1,88% )
    7 519 563 092 instructions              #   2,05  insn per cycle         ( +- 0,02% )
    3 668 765 913 cycles                    #   3,883 GHz                    ( +- 1,92% )
      1 572 770 cache-misses                ( +- 4,13% )
        27 266 page-faults                  # 28,855 K/sec                   ( +- 0,00% )
        27 266 minor-faults                 # 28,855 K/sec                   ( +- 0,00% )
           0 major-faults                   #   0,000 /sec

```

0,9486 +- 0,0176 seconds time elapsed (+- 1,86%)

Вывод:

В ходе выполнения задания №2 («Великий визирь») было показано, что последовательный доступ к данным в отображённом через `mmap( )` файле значительно эффективнее хаотичного: он вызывает меньше промахов кэша и страничных ошибок, особенно при использовании подсказки `MADV_SEQUENTIAL`, которая активирует опережающую подкачку страниц. В то же время режим `MADV_RANDOM` снижает накладные расходы при случайном доступе, а `MADV_WILLNEED` позволяет заранее загрузить данные в память, сокращая количество `major page faults`